


Northern Illinois University

# R-ticulating Data

## A Data Cleaning and Visualization Demo

**Jeremy R. Groves, Ph.D, ACUE**

IEA Conference – October 24, 2025

# Setup

- Do you have R and R-Studio Installed?
  - Create a new project
  - If you have/use GIT, you can clone the repo in project setup, if not, we will get it in a second.

https://github.com/jrgroves/Rticulate

- If no R, use the POSIT cloud where students can log into free and create a project so that they can use the R-Studio interface.

  Go To: https://posit.cloud/

  Create an account (using any email)

  Choose a New R Studio Project from the upper left corner.

# Setup

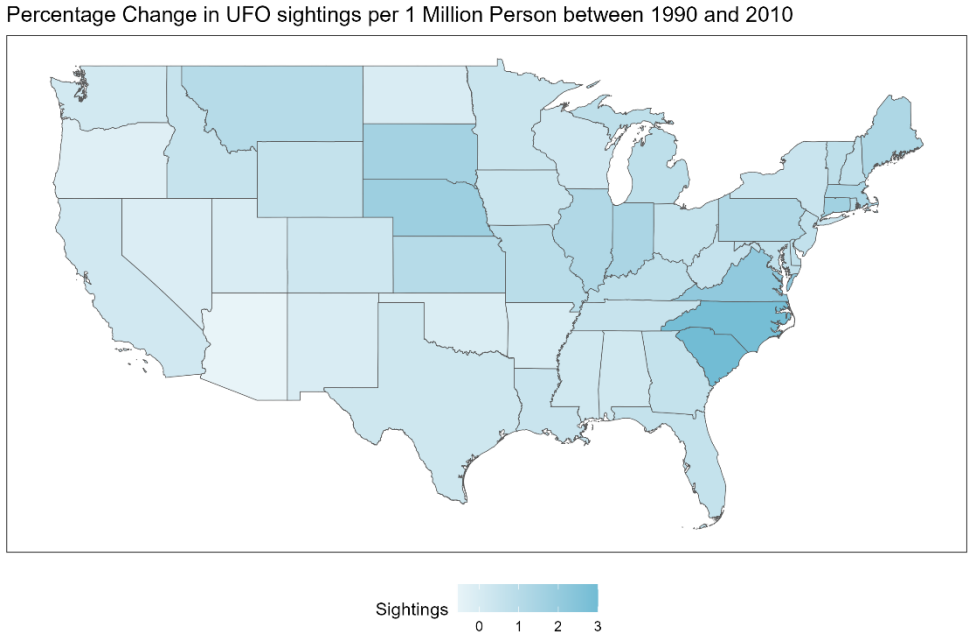
- Type the code below in the console to ensure you have the needed packages installed. Make sure press ENTER after each line.




- This will install the necessary packages for our work.
  - *tidyverse* is a great package for cleaning data and writing scripts
  - *sf* is a package for all our GIS and mapping needs
  - *usethis* is a package we can use for getting our data without GIT.

# Our Mission

- Visualize the percentage change in the number of UFO sightings per person in the continental U.S. between 2000 and 2010.






# A quick detour

Getting the data you need can be the most complicated part of a project. There are several ways to accomplish this task

1. **Original Collection**. This may be going out and measuring something or collecting information or may simply be the transfer of archival data from print to digital formats.

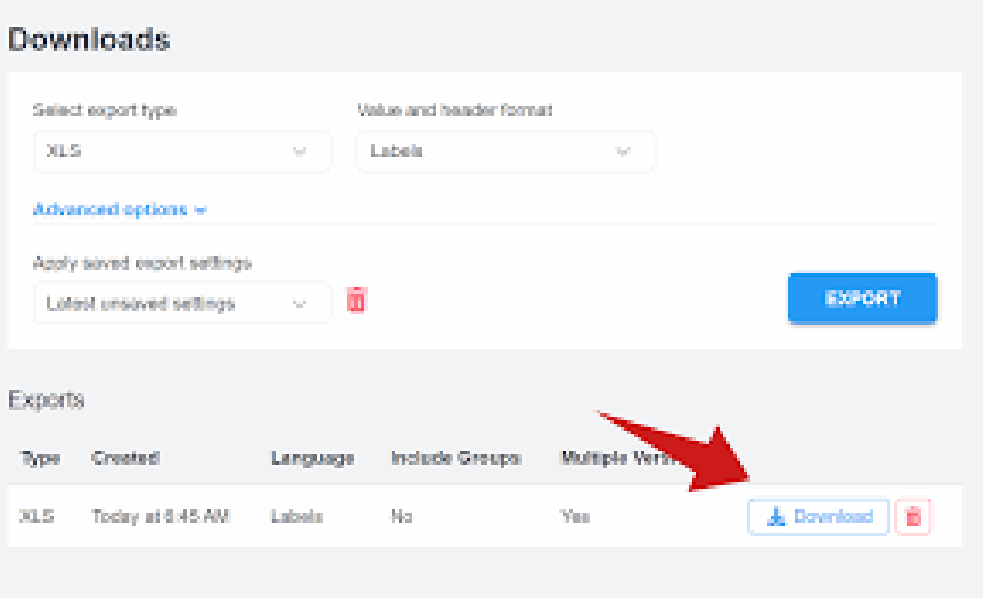


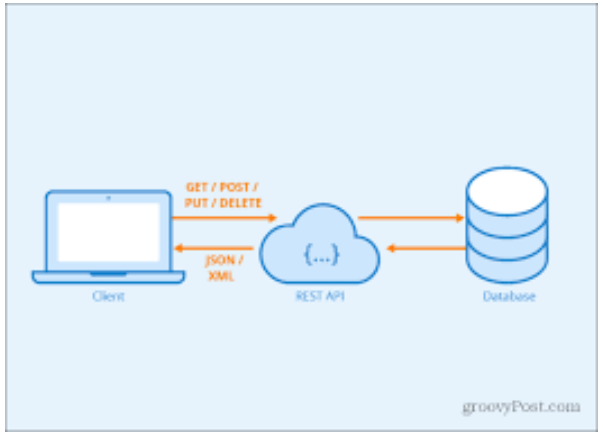




# A quick detour

2. **Downloads:** This can include data from a primary source such as Redfin or Zillow for housing data, from an API source such as a government or industry database/website, or from cloud storage such as OneDrive or GitHub.

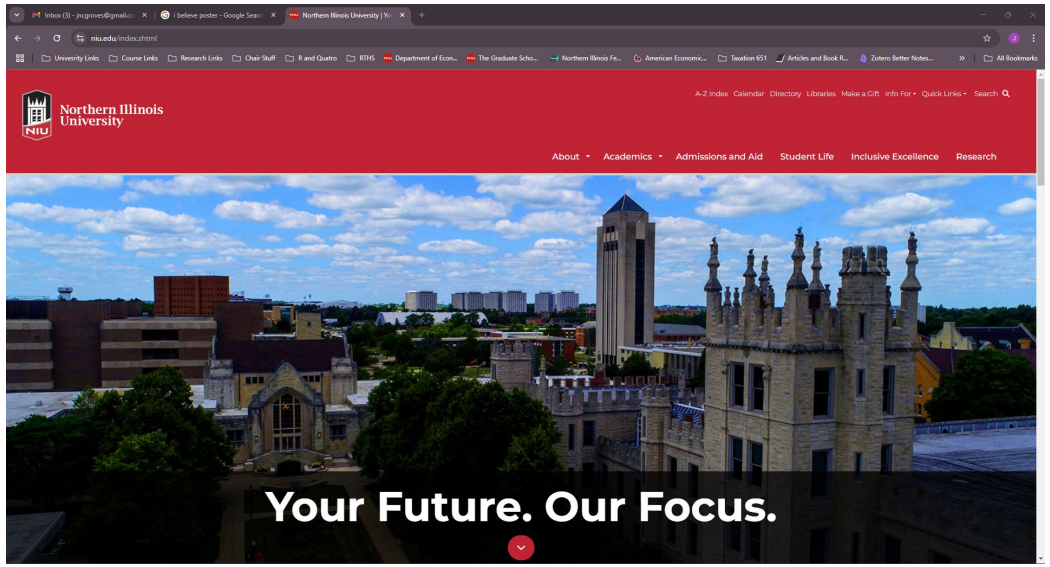





# A quick detour

3. **Scrape It:** Web-scaping is the process of exploiting the fact that websites, as pretty as they might be, are still just a code written on a system according to a language.






```
        <nav class="navbar navbar-global" role="navigation" aria-label="Global">
<div class="container" style="position: relative;">
    <!-- Collect the nav links, forms, and other content for toggling -->
    <ul class="nav justify-content-start">
        <li class="nav-item small">
            <a href="https://www.niu.edu">
                <i class="fal fa-long-arrow-left"></i> NIU.edu
            </a>
        </li>
    </ul>
    <ul class="nav justify-content-end d-none d-md-inline-flex mr-lg-0">
        <li class="nav-item small">
            <a class="nav-link" href="https://www.niu.edu/web.shtml">A-Z Index</a>
        </li>
        <li class="nav-item small">
            <a class="nav-link" href="https://calendar.niu.edu/">Calendar</a>
        </li>
        <li class="nav-item small">
            <a class="nav-link" href="https://directory.niu.edu/app/directory/PeopleSearch.aspx">Directory</a>
        </li>
        <li class="nav-item small">
            <a class="nav-link" href="https://library.niu.edu/ulib/">Libraries</a>
        </li>
        <li class="nav-item small">
            <a class="nav-link" href="https://foundation.myniu.com/">Make a Gift</a>
        </li>
        <li class="nav-item small dropdown">
            <a aria-expanded="false" aria-haspopup="true" class="nav-link dropdown-toggle" data-toggle="dropdown" href="#" role="button">Info For</a>
            <div class="dropdown-menu">
                <a class="dropdown-item" href="https://www.niu.edu/students.shtml">Students</a>
                <a class="dropdown-item" href="https://www.niu.edu/hrs/resources/index.shtml">Employees</a>
                <a class="dropdown-item" href="https://niu.edu/parents.shtml">Parents and Families</a>
                <a class="dropdown-item" href="https://www.myniu.com/" rel="noopener noreferrer">Alumni and Friends</a>
                <a class="dropdown-item" href="https://niu.edu/community/index.shtml">Community and Business</a>
                <a class="dropdown-item" href="https://employment.niu.edu/" rel="noopener noreferrer">Prospective Employees</a>
            </div>
        </li>
        <li class="nav-item small dropdown">
            <a aria-expanded="false" aria-haspopup="true" class="nav-link dropdown-toggle" data-toggle="dropdown" href="#" role="button">Quick Links</a>
            <div class="dropdown-menu">
                <a class="dropdown-item" href="https://webcourses.niu.edu/" rel="noopener noreferrer">Blackboard</a>
                <a class="dropdown-item" href="https://myniu.niu.edu/ps/signon.html" rel="noopener noreferrer">MyNIU</a>
                <a class="dropdown-item" href="https://go.niu.edu/my-scholarships" rel="noopener noreferrer">MyScholarships</a>
                <a class="dropdown-item" href="https://go.niu.edu/get-involved" rel="noopener noreferrer">Huskie Hub</a>
                <a class="dropdown-item" href="https://o365.niu.edu" rel="noopener noreferrer">Office 365</a>
                <a class="dropdown-item" href="https://anywhereapps.niu.edu" rel="noopener noreferrer">Anywhere Apps</a>
                <a class="dropdown-item" href="https://niu.qualtrics.com" rel="noopener noreferrer">Qualtrics</a>
                <a class="dropdown-item" href="https://go.niu.edu/navigate" rel="noopener noreferrer">Navigate</a>
```

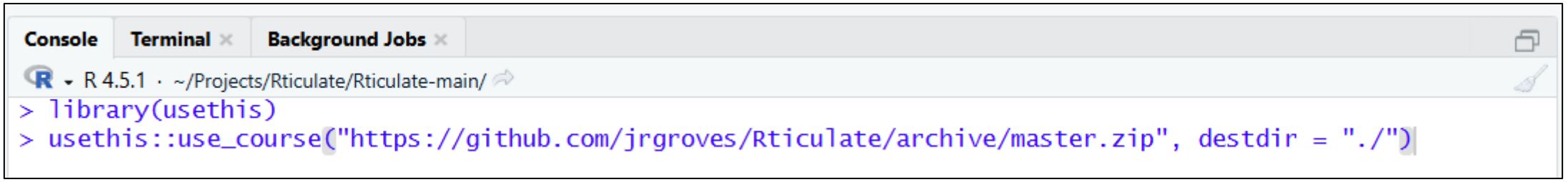
# Our Data

- We get our data from three primary sources. For time saving, it is already going to be in the reop or zip file you download.

1. **UFO Sightings:** National UFO Reporting Center (https://nuforc.org/) and Kraggle (https://www.kaggle.com/datasets/NUFORC/ufo-sightings)
2. **USA Map:** U.S. Census Bureau (https://census.gov) Tiger Line Files (via Census API)
3. **Population Data:** IPUMS NHGIS (via IMPUS API) (https://www.ipums.org/projects/ipums-nhgis)

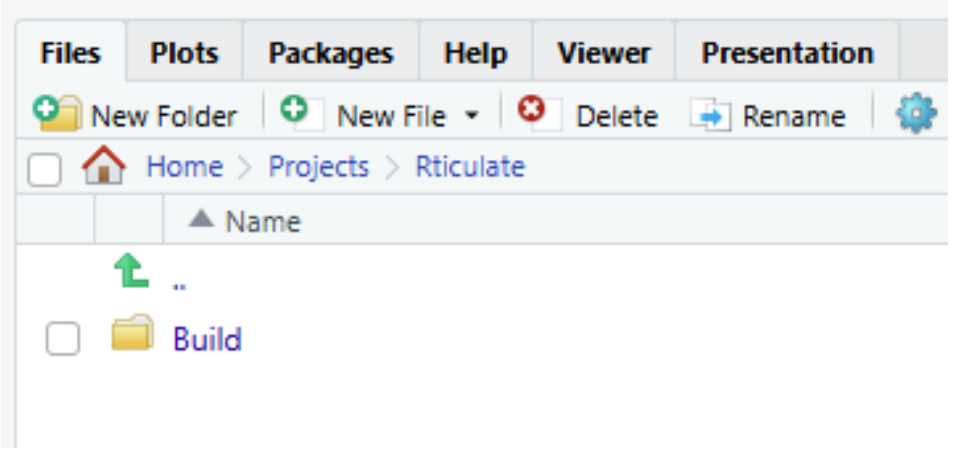
# Getting Our Data

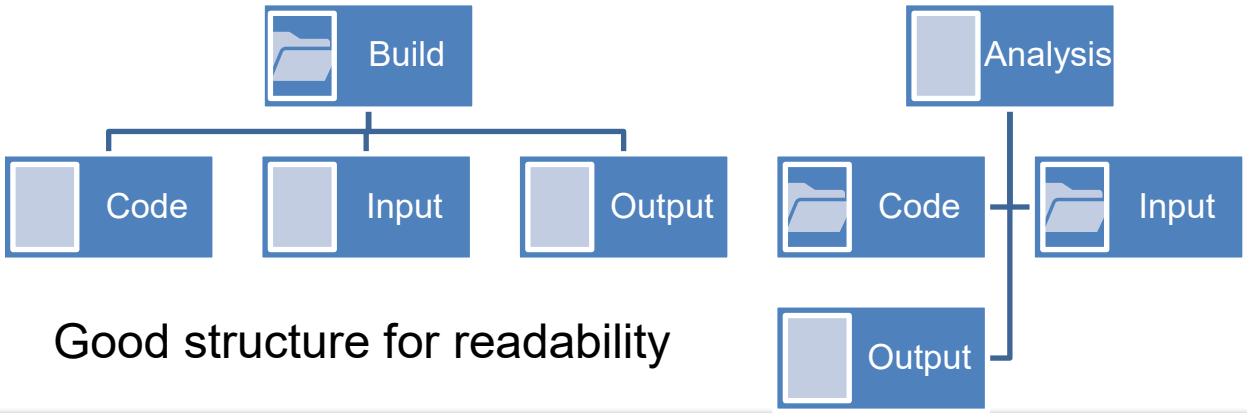
- **Option 1: Clone the GIT Repository**
  - If you created your project using version control, the data was cloned in with the setup.
- **Option 2: Use the `use_course()` command to download**
  - This will allow you to download the makeup of the GIT REPO and it will install into whatever your current directory is.
  - This will likely open a new session of RS, but that is fine.



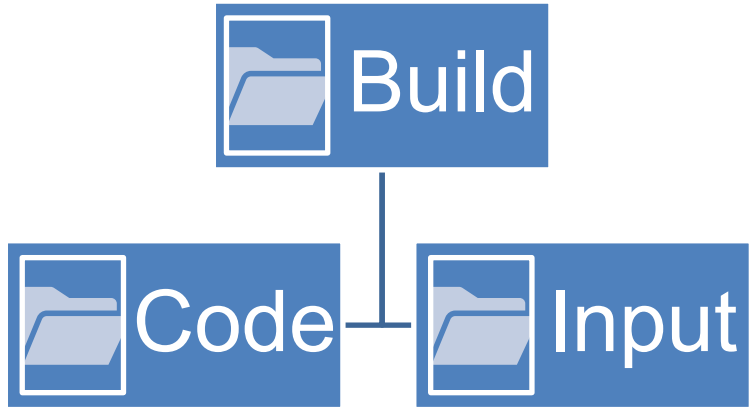
# Finding our Data

- **File Manager in RStudio (RS)**
  - In the lower-right pane in RS, you will find a FILES tab. This will show you the files and folders in your current working directory.
  - We have set up our file structure all under the “Build” folder




Good structure for readability



# Scripting

- With any project it is important to keep track of what you are doing and design it in such as way as to make it reproducible.
- Open a new script using the  in the upper-left corner of RStudio.
- Start by adding a title and description, clean our workspace, and load our libraries.



# Loading in our data

- Once we have our libraries loaded, we need to put the data we will use in our Environment.




- This will read the text file of UFO sightings and two pre-made Rdata files that we created using the APIs *{the code is in the Build Data.R script}.*
- Note that this code assigns each dataframe to the object that is located at the start of the line *{R is an objected orientated language, so everything is an object of some type}*

# Viewing the Mess

- We can use the `head()` command to see what our data looks like. You can type the command directly in the console if you like.

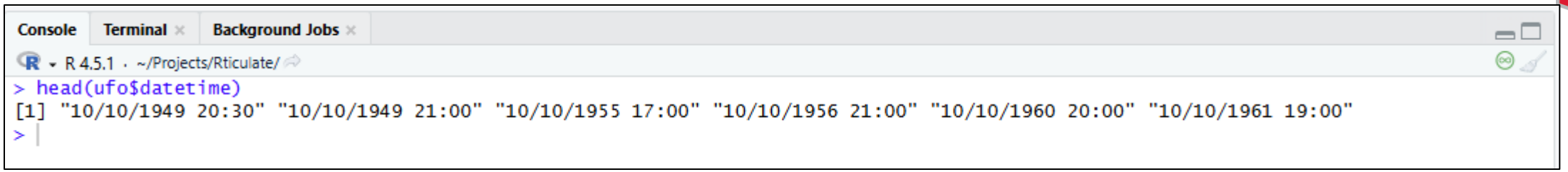
Console | Terminal | Background Jobs

R 4.5.1 · ~/Projects/Rticulate/

```
> head(acs)
Simple feature collection with 6 features and 5 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -124.4096 ymin: 30.22333 xmax: -80.84038 ymax: 45.94545
Geodetic CRS:  NAD83
  GEOID        NAME    variable estimate   moe                       geometry
1    35  New Mexico B01001A_001  1467849  6135 MULTIPOLYGON (((-109.0502 3...
2    46 South Dakota B01001A_001   735228  1110 MULTIPOLYGON (((-104.0579 4...
3    06  California B01001A_001 22053721 34243 MULTIPOLYGON (((-118.6044 3...
4    21    Kentucky B01001A_001  3848305  3113 MULTIPOLYGON (((-89.41728 3...
5    01     Alabama B01001A_001  3302834  3712 MULTIPOLYGON (((-88.05338 3...
6    13     Georgia B01001A_001  6020265 10140 MULTIPOLYGON (((-81.27939 3...
>
```

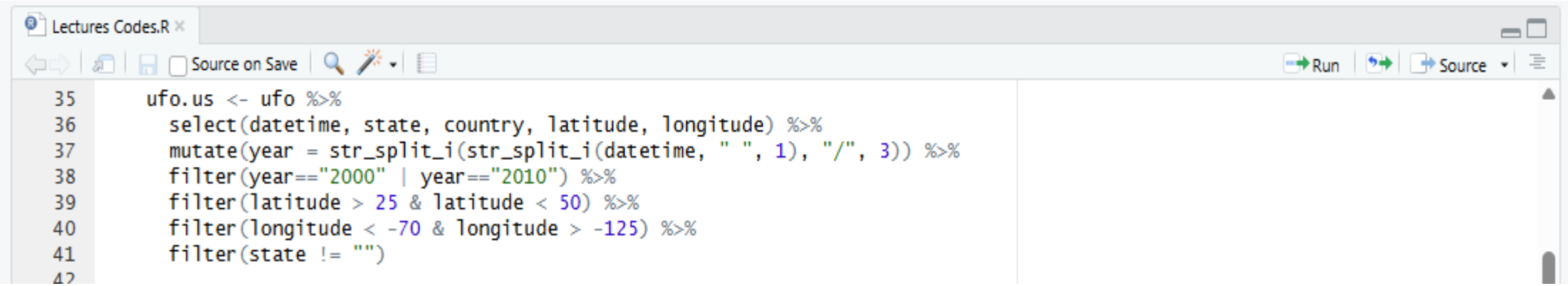
- This is an ‘sf’ object meaning that is designed for geographic work *{see the projection information}* and the ‘geometry’ column which contains the coordinates for the shapes *{state outlines}*.

# Viewing the Mess




- Quickly note that we can view the head of a specific column using the df$colname syntax in the `head()` command.
- This is where the package *tidyverse* shines because we can easily remove, manipulate, filter, and reorder our dataframe all in a single command.
- To do this we use the 'pipped' code to "nest" commands.[ **%>%** ]

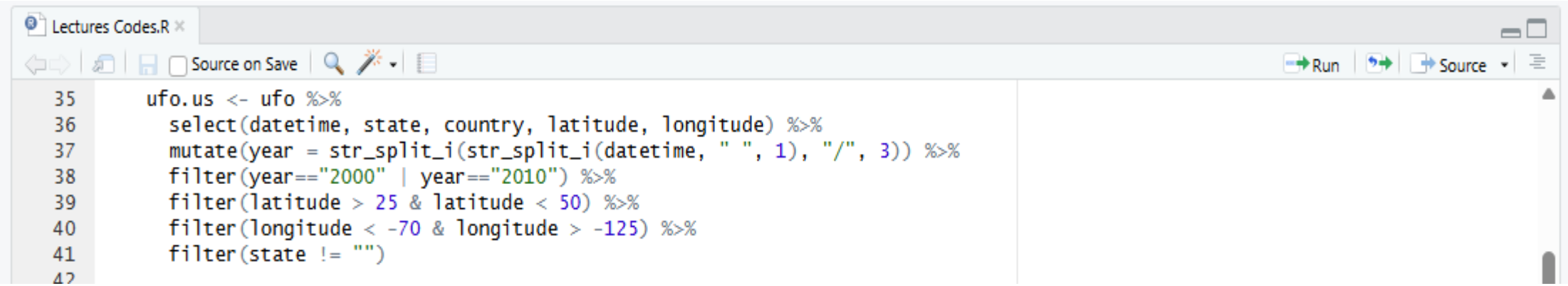
# Scripting our cleanup




- We do quite a bit here and will discuss each line
  35. Assign our new data to a new object (ufo.us) and then setting what dataframe we start with (ufo).
  36. We limit our dataframe to only the columns (vectors) we need
  37. We create a new vector called “year” and then split up the character string that is currently our datetime vector to extract the year.

```
str_split_i(column name, what we split on, part we want to keep)
```

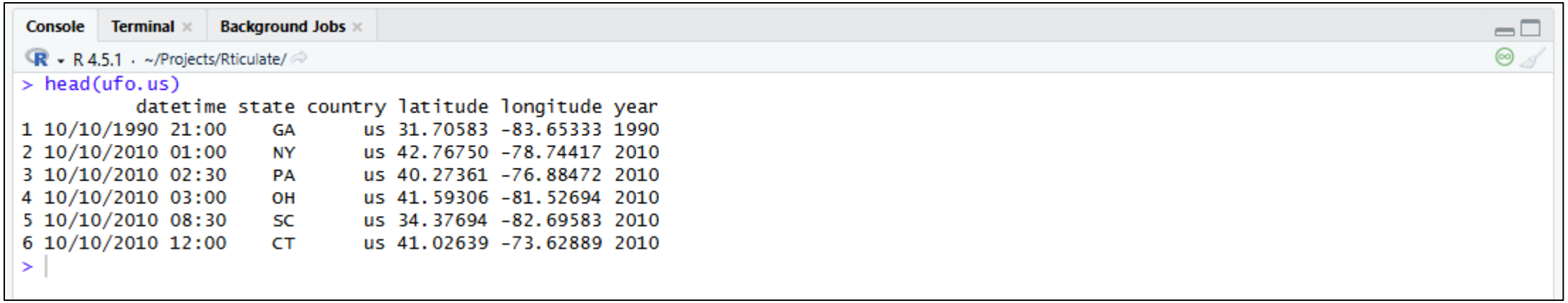
# Scripting our cleanup




38. Keep only sightings that occurred in 2000 and 2010 using the filter command *{and logic operator | which is "or"}*.
39. Remove anything outside the longitude which we can attribute to the US {this is one of those things you find by trial and error}
40. Remove any item that is missing the state data.

# Check In

- To see what we have and to make sure we are progressing, we will look at the head of our new object.




- We see that we have a single observation for each sighting, the abbreviation for the state it occurred and the year it occurred along with some other things we no longer need.
- This type of data is called *LONG DATA* but we want *WIDE DATA*.

# Long and Wide




**Long Format**

| term | date | rate |
|---|---|---|
| 3M | 2021-01-29 | -0.625 |
| 3M | 2021-02-26 | -0.612 |
| 3M | 2021-03-31 | -0.636 |
| 3M | 2021-04-30 | -0.628 |
| 3M | 2021-05-31 | -0.632 |
| 3M | 2021-06-30 | -0.650 |
| 3M | 2021-07-30 | -0.663 |
| 3M | 2021-08-31 | -0.676 |
| 3M | 2021-09-30 | -0.712 |
| 3M | 2021-10-29 | -0.736 |
| 3M | 2021-11-30 | -0.895 |
| 3M | 2021-12-31 | -0.731 |
| 3Y | 2021-01-29 | -0.771 |
| 3Y | 2021-02-26 | -0.648 |
| 3Y | 2021-03-31 | -0.711 |
| 3Y | 2021-04-30 | -0.684 |
| 3Y | 2021-05-31 | -0.666 |
| 3Y | 2021-06-30 | -0.672 |
| 3Y | 2021-07-30 | -0.813 |
| 3Y | 2021-08-31 | -0.760 |
| 3Y | 2021-09-30 | -0.677 |
| 3Y | 2021-10-29 | -0.537 |
| 3Y | 2021-11-30 | -0.766 |
| 3Y | 2021-12-31 | -0.620 |
| 10Y | 2021-01-29 | -0.512 |
| 10Y | 2021-02-26 | -0.246 |
| 10Y | 2021-03-31 | -0.279 |
| 10Y | 2021-04-30 | -0.180 |
| 10Y | 2021-05-31 | -0.146 |
| 10Y | 2021-06-30 | -0.203 |
| 10Y | 2021-07-30 | -0.440 |
| 10Y | 2021-08-31 | -0.393 |
| 10Y | 2021-09-30 | -0.170 |
| 10Y | 2021-10-29 | -0.069 |
| 10Y | 2021-11-30 | -0.350 |
| 10Y | 2021-12-31 | -0.188 |
| 20Y | 2021-01-29 | -0.176 |
| 20Y | 2021-02-26 | 0.103 |
| 20Y | 2021-03-31 | 0.142 |
| 20Y | 2021-04-30 | 0.252 |
| 20Y | 2021-05-31 | 0.287 |
| 20Y | 2021-06-30 | 0.201 |
| 20Y | 2021-07-30 | -0.059 |
| 20Y | 2021-08-31 | -0.033 |
| 20Y | 2021-09-30 | 0.195 |
| 20Y | 2021-10-29 | 0.103 |
| 20Y | 2021-11-30 | -0.115 |
| 20Y | 2021-12-31 | 0.056 |

**Wide Format**

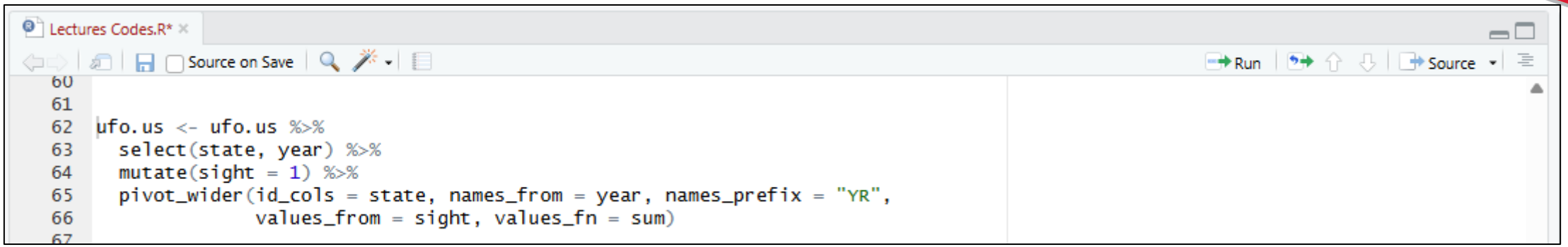
| date | 3M | 3Y | 10Y | 20Y |
|---|---|---|---|---|
| 2021-01-29 | -0.625 | -0.771 | -0.512 | -0.176 |
| 2021-02-26 | -0.612 | -0.648 | -0.246 | 0.103 |
| 2021-03-31 | -0.636 | -0.711 | -0.279 | 0.142 |
| 2021-04-30 | -0.628 | -0.684 | -0.180 | 0.252 |
| 2021-05-31 | -0.632 | -0.666 | -0.146 | 0.287 |
| 2021-06-30 | -0.650 | -0.672 | -0.203 | 0.201 |
| 2021-07-30 | -0.663 | -0.813 | -0.440 | -0.059 |
| 2021-08-31 | -0.676 | -0.760 | -0.393 | -0.033 |
| 2021-09-30 | -0.712 | -0.677 | -0.170 | 0.195 |
| 2021-10-29 | -0.736 | -0.537 | -0.069 | 0.103 |
| 2021-11-30 | -0.895 | -0.766 | -0.350 | -0.115 |
| 2021-12-31 | -0.731 | -0.620 | -0.188 | 0.056 |

- Think of moving from long to wide as an aggregation.
- In our case, we want to aggregate the number of sightings by state and by year.
- There are several ways we can do this, but we are going to use the set of pivot commands in *tidyverse*.

```
pivot_wider(id_cols, names_from,
            names_prefix, values_from, values_fn)
```

  - **id_cols:** This is the name of the vector(s) that is our key identifier.
  - **names_from:** What each new vector is going to be named (the grouping)
  - **names_prefix:** R does not allow vector names to start with a number.
  - **values_from:** What vector are we going to aggregate
  - **value_fn:** How do we aggregate that vector?

# Scripting our Cleanup




- We update by our existing dataframe by simply assigning it to the same object name as we did before **{warning: make sure you want to do this}**
  62. Assign our object name and give our starting dataframe
  63. Keep only the two variables we need
  64. Create a new vector to give the `pivot_wider()` command something to count.
  65. Use the `pivot_wider()` command to group by states and create a count for each year (with the prefix in the column name) and add up (sum) the values from the sight column.

# Scripting our Cleanup

- Use our head command to look at how our data changed




- Rather than having a row for each sighting, we have a row for each state with a column showing the total number of sightings in each state for each year.

# Adding our Potential Abductees

- If we compare our ufo and census data, we will notice we have a bit of an issue.

```
Console  Terminal  Background Jobs
R 4.5.1 · ~/Projects/Rticulate/
> head(ufo.us)
# A tibble: 6 × 3
  state YR2000 YR2010
  <chr>  <dbl>  <dbl>
1 MO        51    104
2 FL       107    204
3 WI        43     58
4 KY        26     50
5 WA       187    252
6 NY       106    158
>
```

```
Console  Terminal  Background Jobs
R 4.5.1 · ~/Projects/Rticulate/
> head(census)
# A tibble: 6 × 5
  STATE      pop_2000 pop_2010 GEOID
  <chr>         <dbl>    <dbl> <chr>
1 Alabama     4447100  4779736 01    (((-88.05338 30.50699, -88.05109 30.50886, -88.04565
2 Alaska       626932   710231 02    (((-160.8657 55.32126, -160.8619 55.32187, -160.8599
3 Arizona     5130632  6392017 04    (((-114.8163 32.50804, -114.8143 32.50902, -114.8102
4 Arkansas    2673400  2915918 05    (((-94.61792 36.49941, -94.61765 36.49941, -94.59595
5 California 33871648 37253956 06    (((-118.6044 33.47855, -118.5988 33.47794, -118.5859
6 Colorado    4301261  5029196 08    (((-109.0603 38.59933, -109.0595 38.71989, -109.0587
>
```

- The census data is listed by state name or GEOID while our UFO data is listed by state abbreviation. There are a few ways to deal with this:
  - We can create a text file to “bridge” the two dataframes.
  - We can create a new column with either abbreviations or GEOIDs in the UFO data.

# Load and Merge / Join

- When we have several different datasets, we can merge or join them based on common elements. In *tidyverse* there are three main ways to join data.
  - `left_join(df1, df2)`: this will join two datasets keeping all matched pairs and any unmatched objects from the FIRST dataframe.
  - `right_join(df1, df2)`: this will join two datasets keeping all matched pairs and any unmatched objects from the SECOND dataframe.
  - `full_join(df1, df2)`: this will join two datasets keeping all matched pairs and any unmatched objects from BOTH dataframes.
- Another advantage of the commands in *tidyverse* we can join two dataframes even if the column names are different.

# Load and Merge / Join

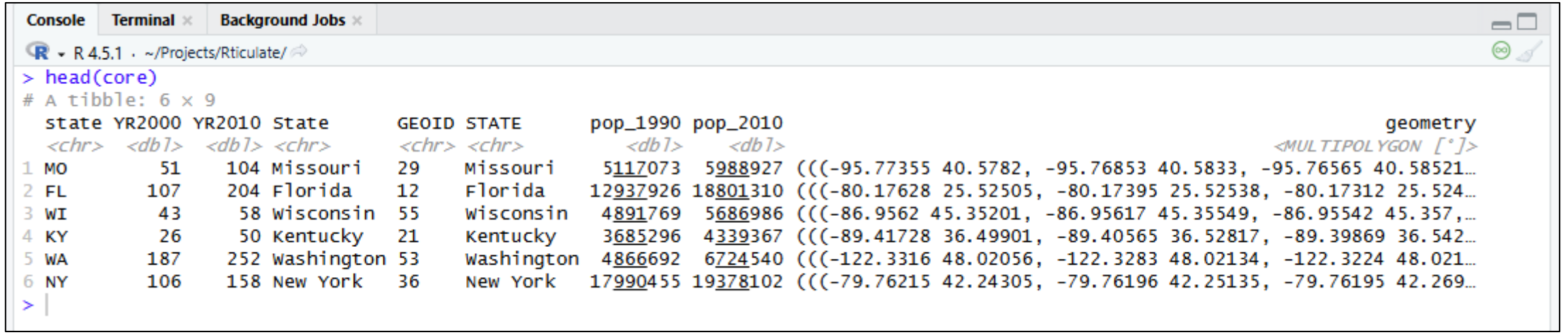



71. Loads the bridge file we previously created.

73. Create our new dataframe
74. Join with the bridge file keeping all elements in the ufo.us dataframe and discarding any unmatched from the bridge file. Joining by the state and Abbr vectors.
75. Join with the census file keeping all elements of our newly joined file from 74 and discarding any unmatched from the census file. Join on the common element GEOID.
76. Filter out any cases where State (which is from census file) has missing data.

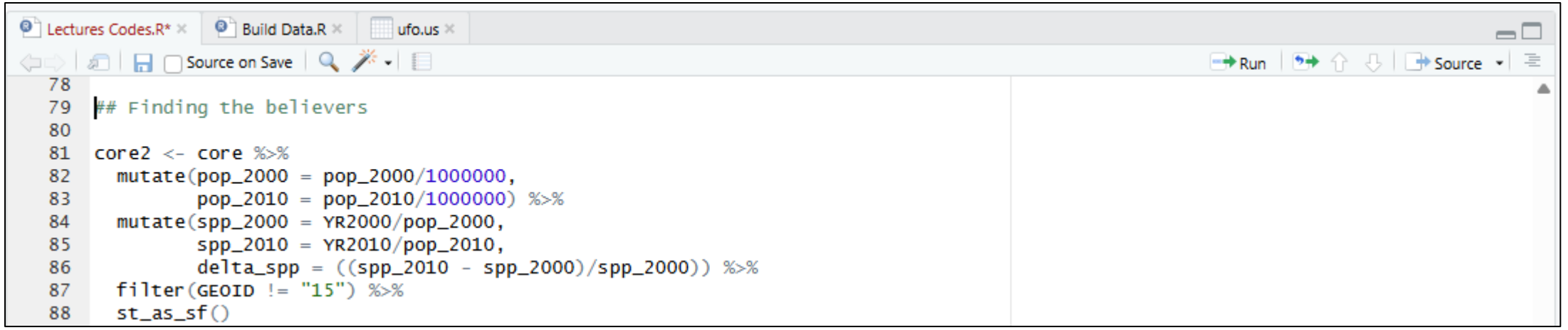
# Load and Merge / Join



# Creating what we want to show

- Now that we have brought together data that likely never thought they would ever meet, we want to massage it into something we can display.



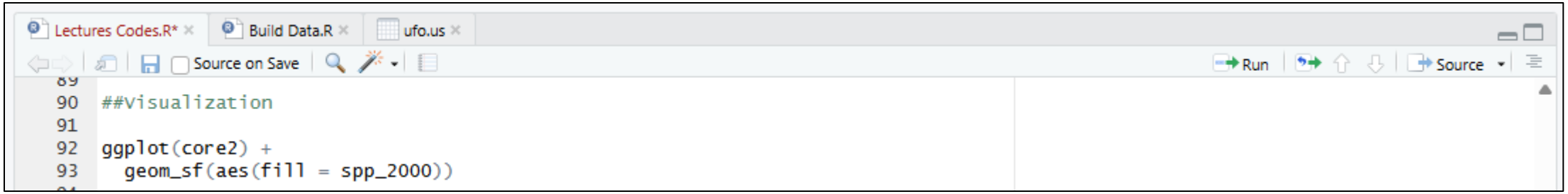

82. Scale our population measures to ‘per one million persons’ so we can show numbers that make a bit of sense.
83. Divide sightings by the adjusted population
84. Calculate the percentage change between 2000 and 2010.
85. Get rid of Alaska
86. Reset our new object as an ‘sf’ object so we can plot it.

# Visualization of Data

- Whether you are a full professor or still a student, one of the key elements of our job is to be able to convey ideas to fellow professionals (both within and outside economics) and the general population the results of our work.
- Believe it or not, the general population does not understand statistics.
- Tables are nice but can take too long to get across an idea.
- Good visualization can also help researchers ask new questions because "seeing" something in a picture can be very different than "reading" something in a table.
- There are two general packages for visualizing data:
  - *ggplot2* (part of tidyverse)
  - *gt* and *gtsummary* (tables)

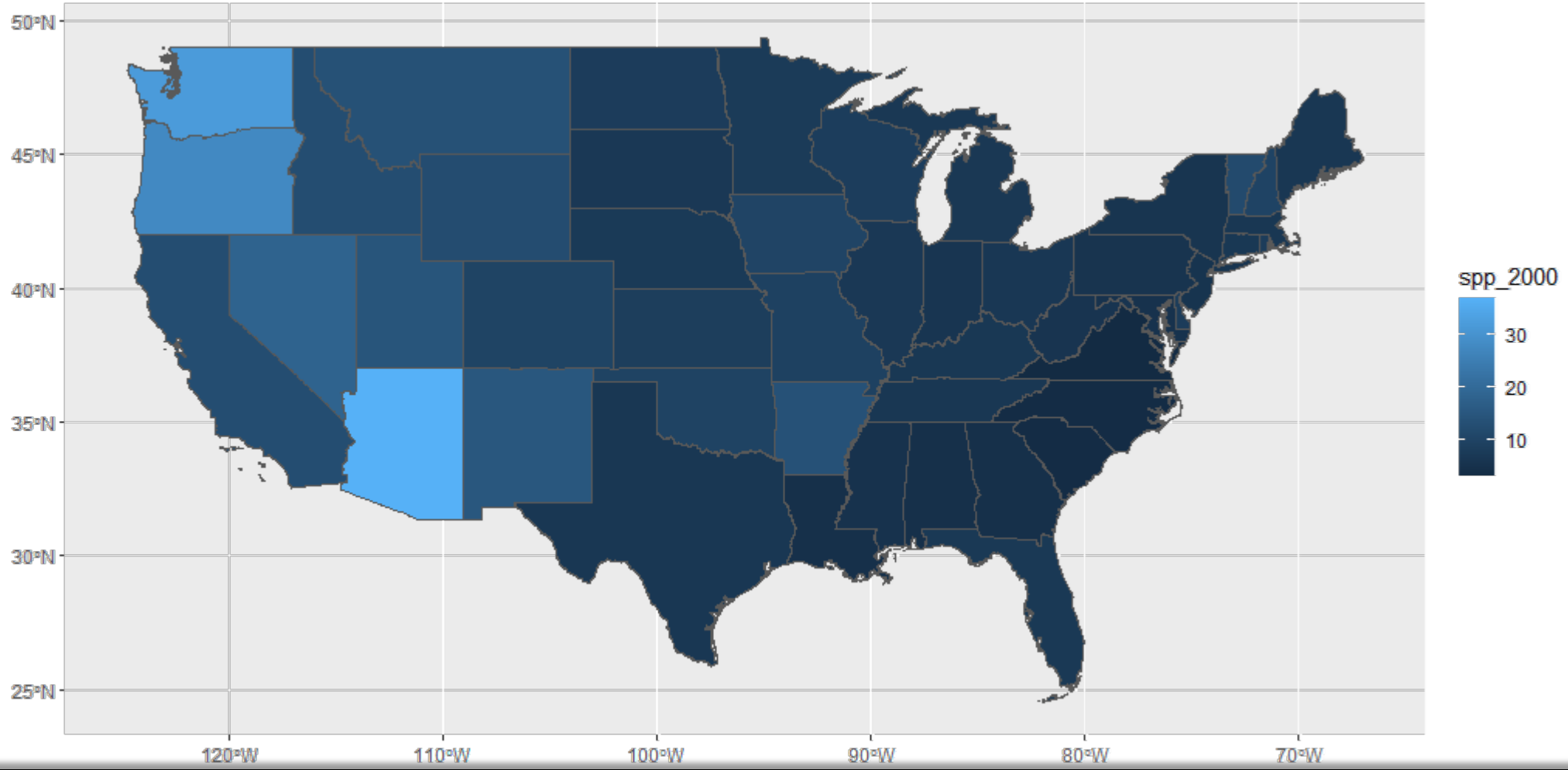
# Working with ggplot2

- Scripting order
  - Start with calling the function [`ggplot()`]and then assign any constants (like data source).
  - Start adding plot layers (geoms) from the bottom up (each line is placed above the last plot) setting their key elements with the aesthetics command [`aes()`]
  - Modify the plot space where the graph is shown



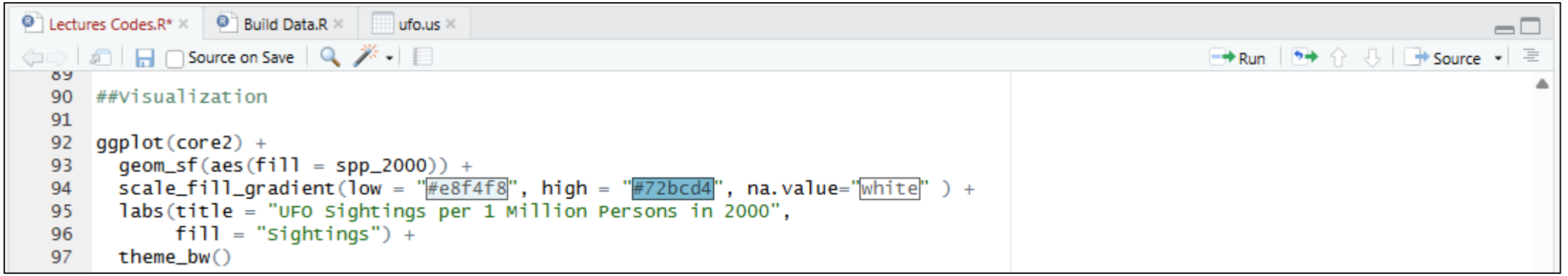

92. Call the function and assign data source as **core2** dataframe.
93. Add our first layer of an 'sf' object and define the aesthetics of how to fill the states

# Working with ggplot2



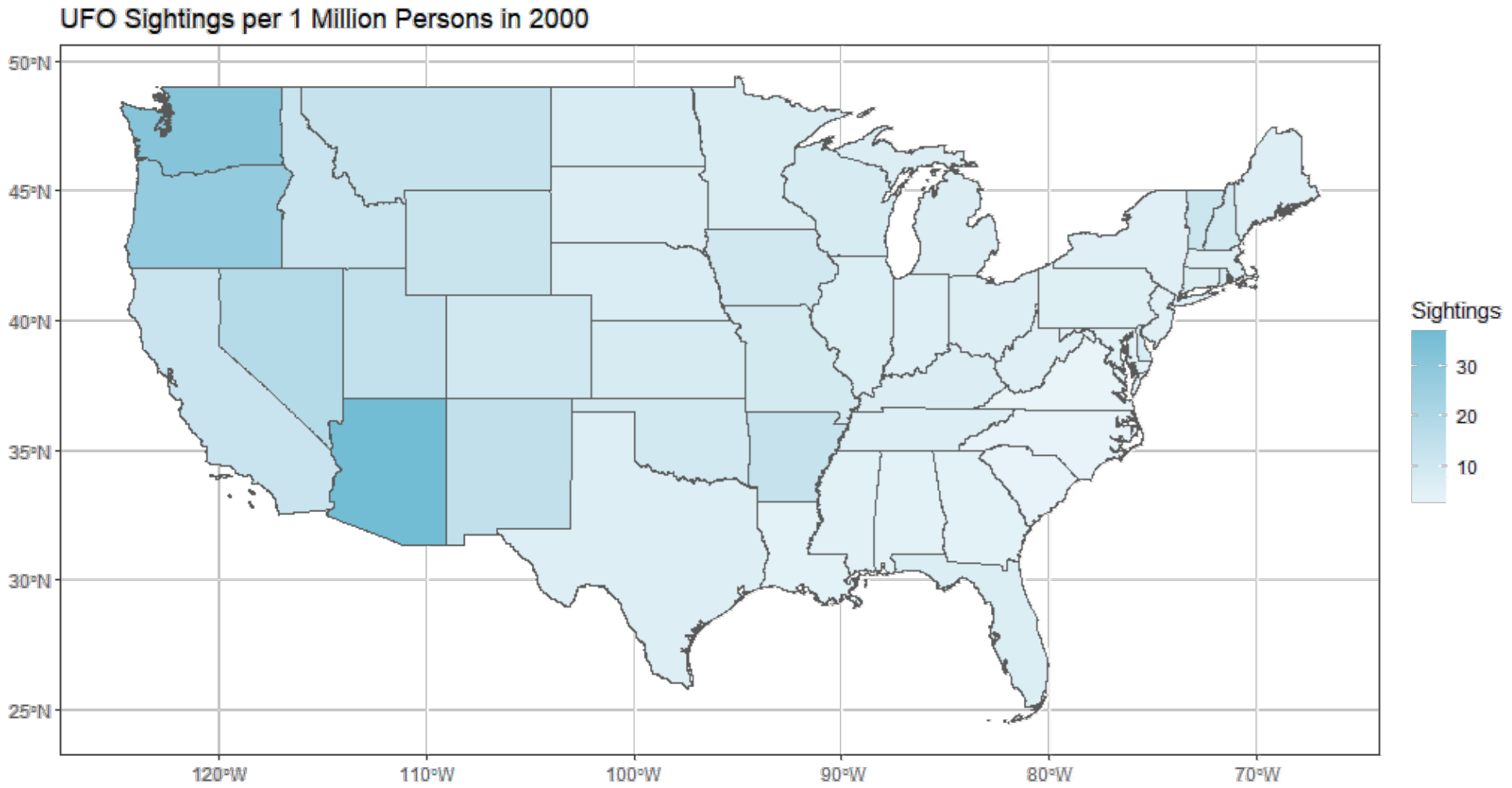
# Working with ggplot2

- Once we have the key elements on the plot, we can start our customization {google is a GREAT resource for ideas and troubleshooting}




92. Call the function and assign data source as core2 dataframe.
93. Add our first layer of an 'sf' object and define the aesthetics of how to fill the states
94. Create new color ends for our gradient fill with light being low end and darker being high end
95. Label our graph and our legend.
97. Make the background a bit cleaner.

# Working with ggplot2



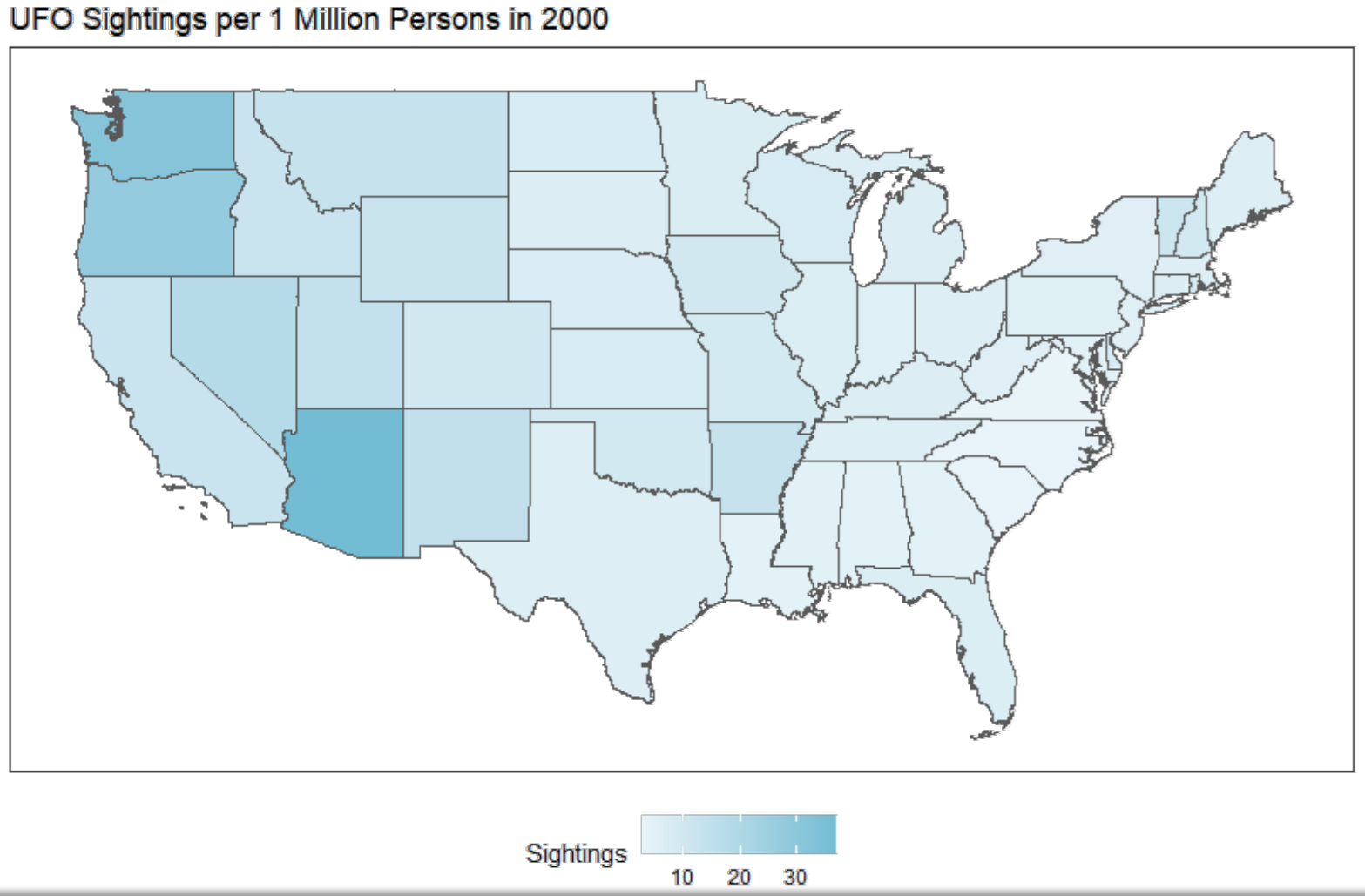
# Working with ggplot2

- The `theme()` command is where we can really get to customizing.

```
 97   theme_bw() +
 98   theme(legend.position="bottom",
 99         axis.text.x = element_blank(),
100         axis.text.y = element_blank(),
101         panel.grid.major = element_blank(),
102         #panel.border = element_blank(),
103         axis.ticks = element_blank())
```

97. Make the background cleaner
98. Move the legend to bottom and make it horizontal
99. Remove the x-axis tick labels
100. Remove the y-axis tick labels
101. Remove grid lines in the plot area
102. Remove the box around the graph (this is commented out so R will ignore it and leave the box).
103. Remove the axis tick marks

# Working with ggplot2



# Working with ggplot2

- We can also put both our graphs in the same picture using another package (*cowplot*)



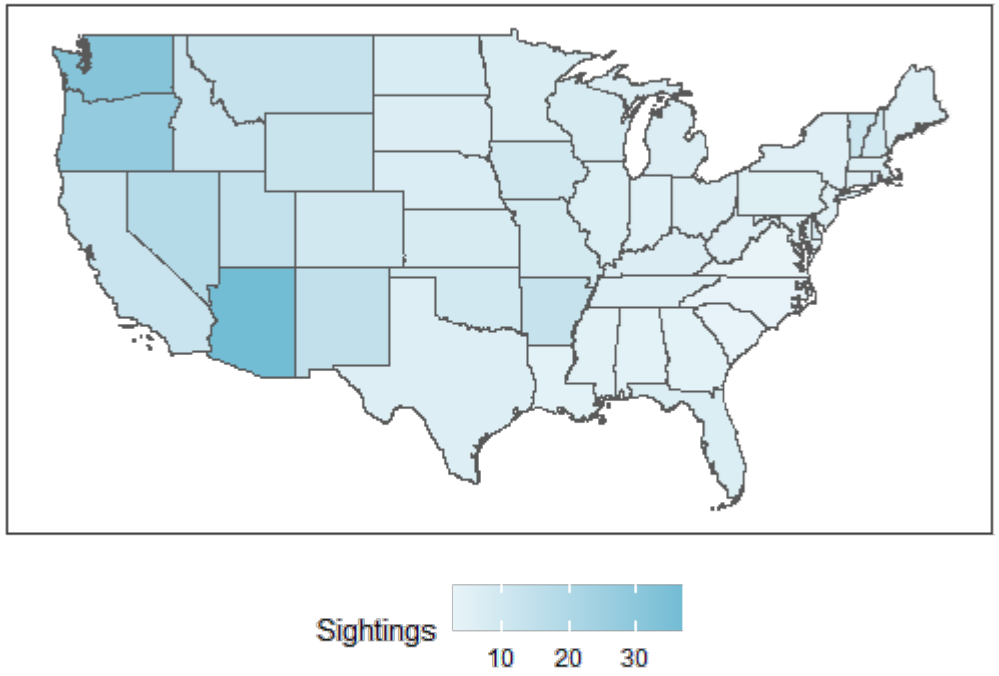

121. Install the package we need
122. Load it into the environment
123. Create a one row grid of plots with the 2000 and 2010 version of the plots we made.

- To assign a plot to an object, we just add the assignment operator before the `ggplot()` command call.

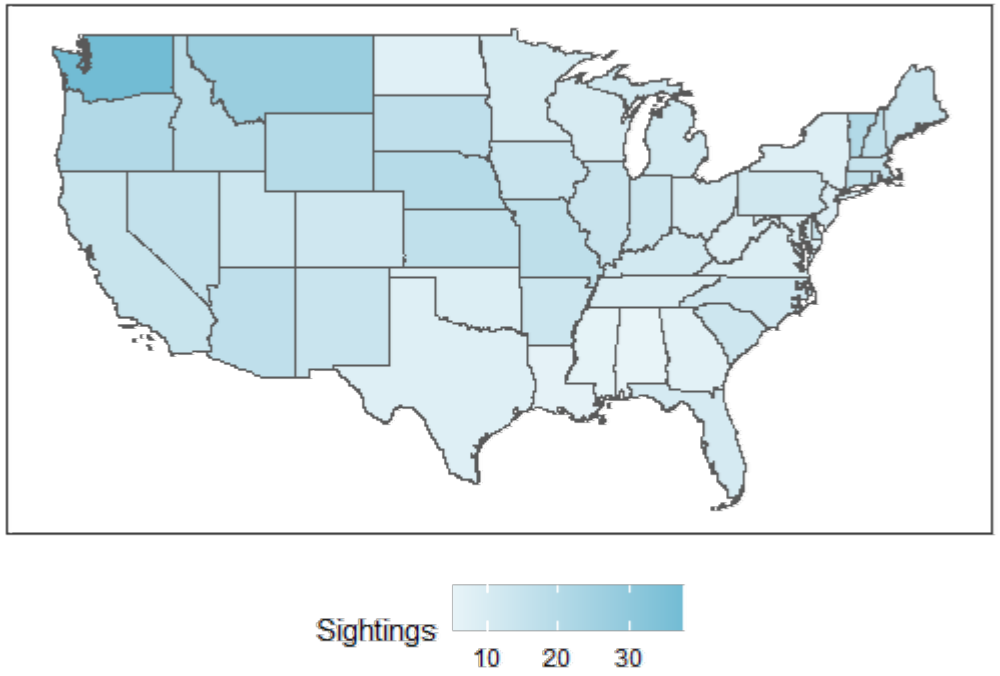
```
g1 ← ggplot(core2) +
```

# Working with ggplot2

UFO Sightings per 1 Million Persons in 2000




UFO Sightings per 1 Million Persons in 2010



# Working with ggplot2

- To display our visualization we are working for, we can just copy the previous code and make some adjustments where needed.

Lectures Codes.R

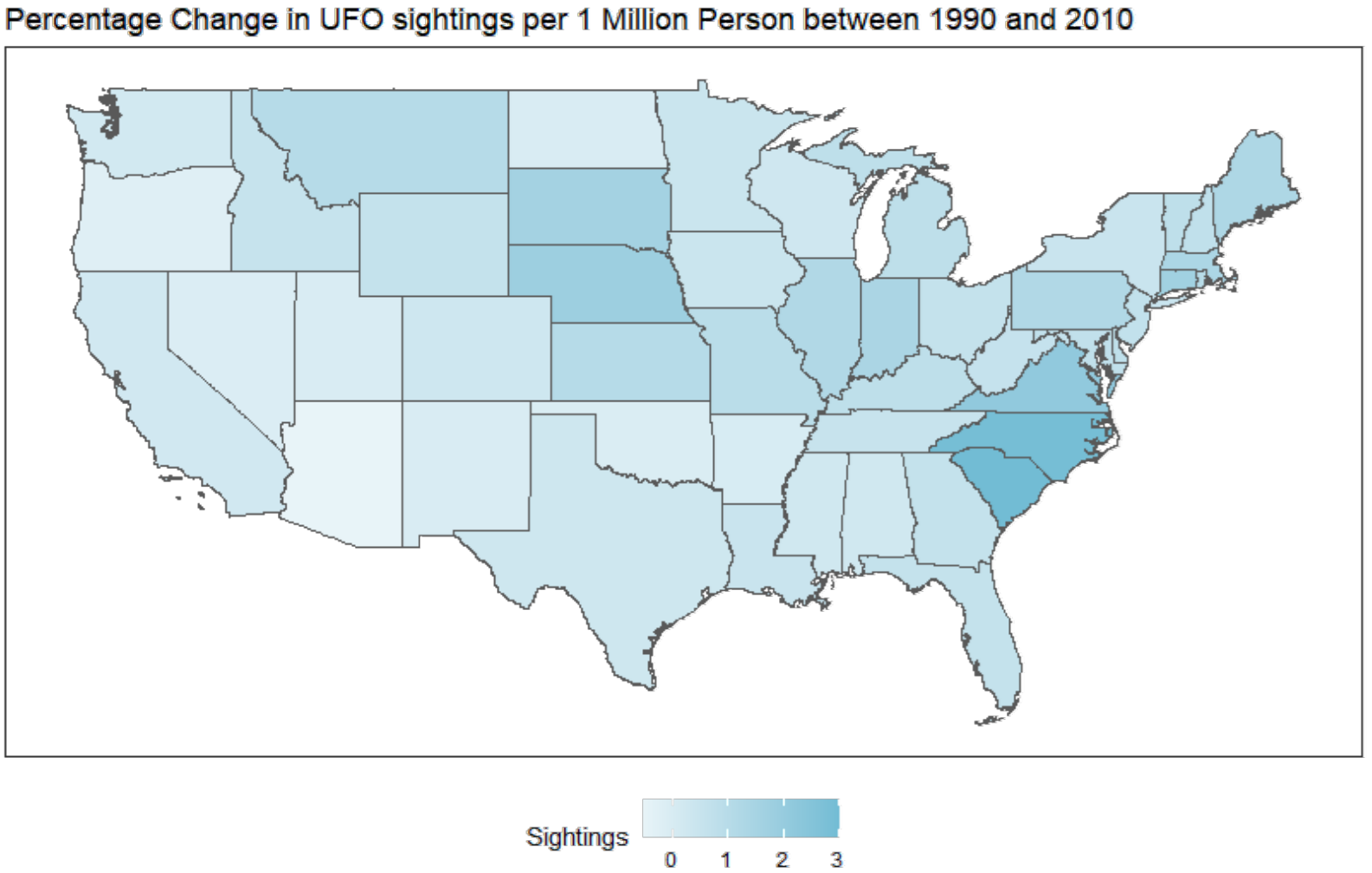
Source on Save | Run | Source

```
## Creation of Percentage Change Plot - more modifications and save
  g3 <- ggplot(core2) +
    geom_sf(aes(fill = delta_spp)) +
    scale_fill_gradient(low = "#e8f4f8", high = "#72bcd4", na.value="white" ) +
    labs(title = "Percentage Change in UFO sightings per 1 Million Person between 1990 and 2010",
         fill = "Sightings") +
    theme_bw() +
    theme(legend.position="bottom",
          axis.text.x = element_blank(),
          axis.text.y = element_blank(),
          panel.grid.major = element_blank(),
          #panel.border = element_blank(),
          axis.ticks = element_blank())

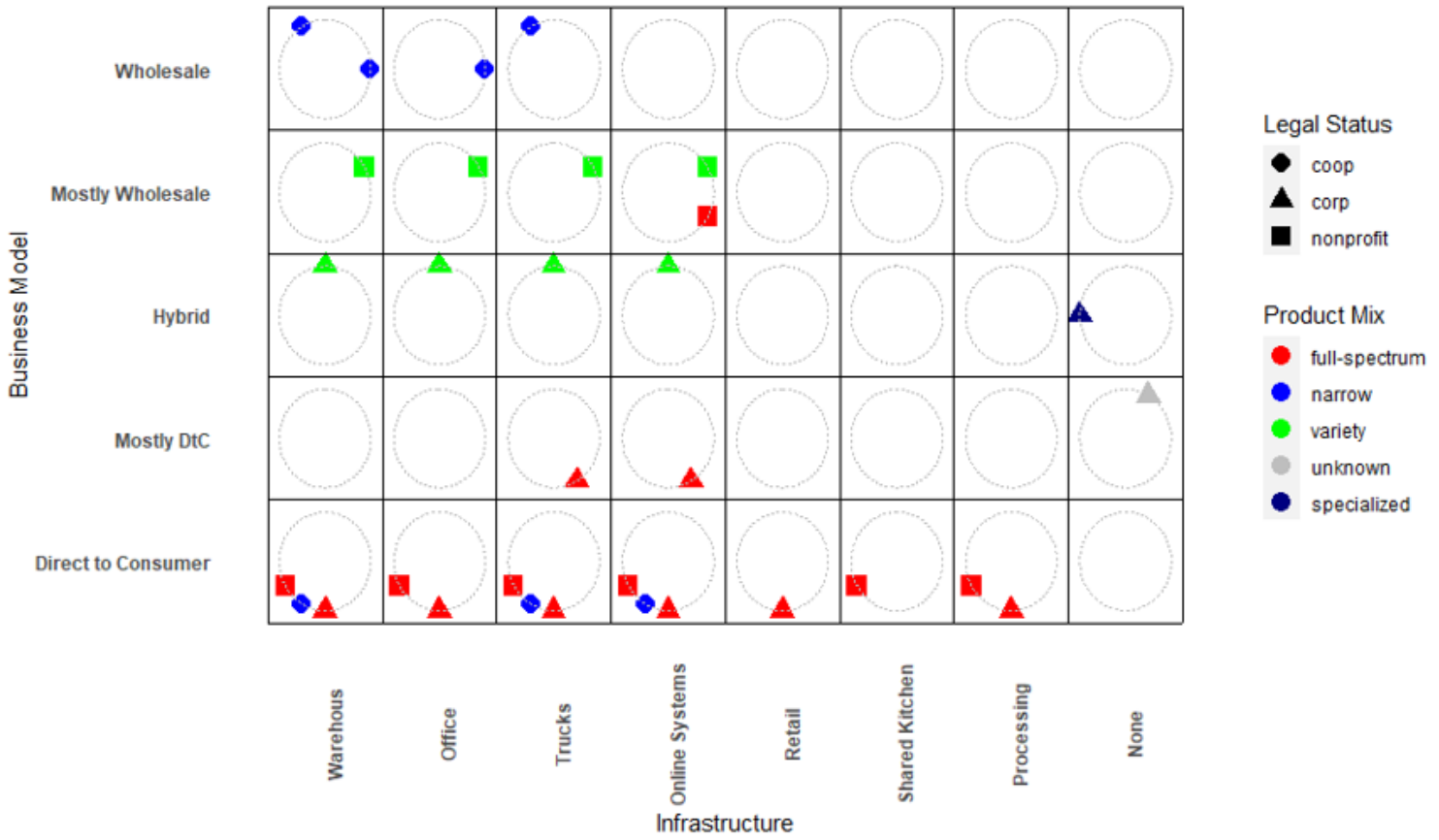
ggsave(g3, file = "./Figure One.png", dpi = 300)
```

- We also add the command to save this graph to our computer as a .png file for addition in a presentation document.

# Working with ggplot2



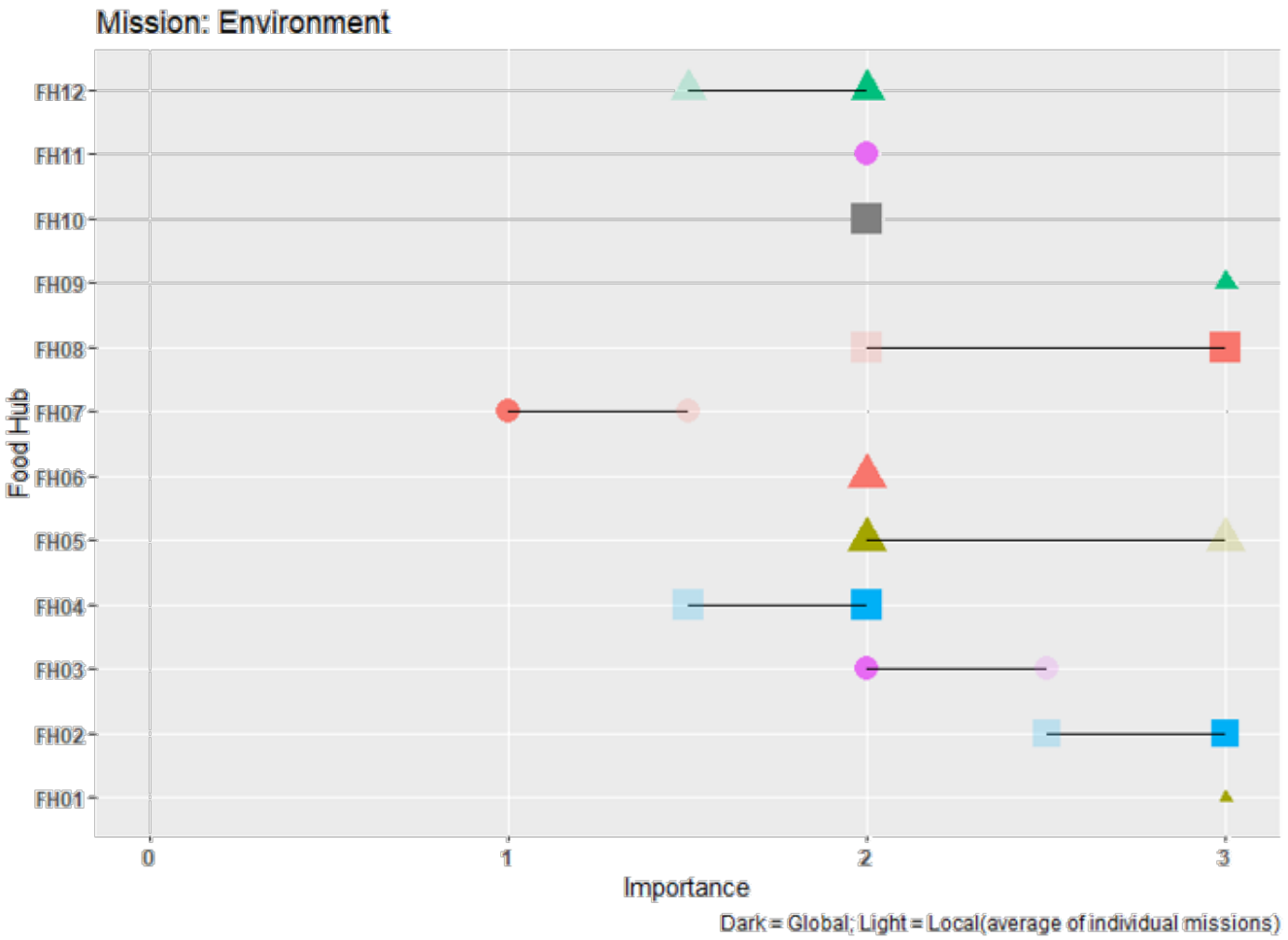
# Create your very own graphics




Plot created for a project with 12 different organizations of different types and classifications and wanted to show their business model and available infrastructure in a single image

Based on the hands of a clock with each organization numbered 1 - 12

# Create your very own graphics







Convey the range of importance of the environment to each of the organizations by type, legal status, and business model

Like a plot of regression coefficients.